

Análisis automatizado de sentencias del Tribunal de Justicia de la Unión Europea

Arquitectura Análítica y Soporte Técnico Profesional (Horizonte
2018-2026

Autor: Argiñe Delgado

Año: 2026

Versión: 1

Resumen

Este trabajo presenta el diseño y desarrollo de un pipeline de procesamiento documental orientado al análisis inicial de sentencias del Tribunal de Justicia de la Unión Europea (TJUE). La herramienta, implementada en Python, permite procesar colecciones de documentos PDF, extraer su contenido textual, detectar referencias a artículos y normas, clasificar las sentencias por categorías temáticas y generar indicadores auxiliares sobre la vigencia doctrinal configurada en el sistema.

Una característica central del proyecto es la optimización del procesamiento. En lugar de volver a abrir y extraer cada documento durante las distintas fases del análisis, la versión realiza una extracción inicial y reutiliza el texto obtenido. Además, el procesamiento de documentos se apoya en `ThreadPoolExecutor`, mientras que la interfaz gráfica se mantiene separada del trabajo pesado mediante una cola de comunicación.

El sistema incorpora una interfaz gráfica desarrollada con Tkinter y genera resultados en HTML, JSON y CSV. Las reglas de clasificación y detección se basan en expresiones regulares y palabras clave, por lo que el sistema es determinista y reproducible, aunque **no sustituye la revisión jurídica humana**.

Palabras clave: TJUE/ procesamiento documental/ Python/ PDF/ jurisprudencia/ expresiones regulares/ clasificación temática/ automatización jurídica.

Abstract

This dissertation presents the design and development of a document-processing pipeline aimed at the initial analysis of judgments of the Court of Justice of the European Union (CJEU). The tool, implemented in Python, processes collections of PDF documents, extracts textual content, detects references to legal provisions and regulations, classifies judgments into thematic categories, and generates auxiliary indicators based on predefined doctrinal-status rules.

A central feature of the project is performance optimization. Instead of reopening and extracting each PDF during every analysis stage, the fast version performs an initial extraction and reuses the resulting text. In addition, document processing relies on `ThreadPoolExecutor`, while the graphical interface remains responsive by separating background processing from the Tkinter event loop through a queue.

The system includes a Tkinter graphical interface and exports results to HTML, JSON and CSV. Classification and detection are based on deterministic regular expressions and keyword rules. Therefore, the system is reproducible and suitable as a document-preprocessing and legal-research support tool, but **it does not replace human legal review**.

Keywords: CJEU, document processing, Python, PDF, case law, regular expressions, thematic classification, legal automation.

Índice general

Resumen	1
Abstract	2
1. Introducción	1
1.1. Contexto	1
1.2. Problema de investigación	1
1.3. Objetivos	2
1.3.1. Objetivo general	2
1.3.2. Objetivos específicos	2
1.4. Alcance	2
2. Estado de la cuestión y marco conceptual	3
2.1. Procesamiento documental jurídico	3
2.2. Expresiones regulares	3
2.3. Clasificación basada en reglas	4
2.4. Paralelización	4
3. Metodología	5
3.1. Diseño experimental	5
3.2. Fase de extracción	5
3.3. Fase de análisis	5
3.4. Precompilación de expresiones	6
3.5. Interfaz gráfica	6
3.6. Exportación	6

4. Diseño e implementación	7
4.1. Arquitectura	7
4.2. Detección de artículos	7
4.3. Detección normativa	8
4.4. Semáforo doctrinal	8
4.5. Resumen automático	8
4.6. Estructura de datos	9
5. Resultados	10
5.1. Resultados funcionales	10
5.2. Resultados de rendimiento	10
5.3. Resultados de clasificación	11
6. Discusión	12
6.1. Ventajas	12
6.2. Limitaciones	12
6.3. Reproducibilidad	13
6.4. Utilidad jurídica	13
7. Conclusiones	14
8. Trabajo futuro	15
A. Anexo A: Fragmentos principales	17
A.1. Extracción de texto	17
A.2. Procesamiento concurrente	17
A.3. Procesamiento desde texto reutilizado	17

Índice de cuadros

2.1. Categorías temáticas configuradas	4
4.1. Semáforo configurado en el sistema	8

Capítulo 1

Introducción

1.1. Contexto

El análisis de jurisprudencia europea implica trabajar con documentos extensos y heterogéneos. Cuando el número de sentencias aumenta, determinadas tareas preliminares —como localizar referencias normativas, agrupar documentos por materias o preparar conjuntos de datos— pueden consumir una cantidad considerable de tiempo.

El proyecto desarrollado aborda esta fase preliminar mediante un pipeline automatizado. La herramienta no pretende determinar de forma autónoma el significado jurídico de una sentencia, sino organizar información documental que posteriormente pueda ser revisada por una persona investigadora o profesional del Derecho.

El archivo de referencia del proyecto es `pipeline_tjue_visual_rapido.py`. Su diseño combina extracción de texto PDF, expresiones regulares, reglas de clasificación, procesamiento concurrente, exportación de datos y una interfaz gráfica.

1.2. Problema de investigación

El problema práctico que motiva el proyecto puede formularse de la siguiente manera:

¿Cómo diseñar una herramienta reproducible y suficientemente rápida que permita procesar una colección de sentencias del TJUE, extraer referencias jurídicas relevantes y organizar los documentos para facilitar su posterior análisis?

La pregunta no se plantea como un problema de interpretación jurídica automática, sino como un problema de ingeniería documental aplicada a la investigación jurídica.

1.3. Objetivos

1.3.1. Objetivo general

Diseñar e implementar un pipeline en Python para el procesamiento automatizado y visual de sentencias del TJUE, priorizando la reutilización del texto extraído, el procesamiento concurrente y la generación de resultados estructurados.

1.3.2. Objetivos específicos

1. Extraer texto de documentos PDF.
2. Detectar referencias a artículos mediante expresiones regulares.
3. Identificar referencias a determinadas normas europeas.
4. Clasificar documentos en categorías temáticas.
5. Generar un indicador auxiliar de estado doctrinal.
6. Procesar documentos de forma concurrente.
7. Mantener la interfaz gráfica responsiva.
8. Exportar los resultados a HTML, JSON y CSV.

1.4. Alcance

El alcance del proyecto comprende el procesamiento de PDFs cuyo contenido textual pueda ser extraído directamente. Los documentos escaneados que no incorporen una capa de texto pueden requerir OCR, funcionalidad que no forma parte del núcleo descrito en este trabajo.

La clasificación temática y el semáforo doctrinal se entienden como mecanismos auxiliares configurables. No se presentan como una valoración jurídica definitiva.

Capítulo 2

Estado de la cuestión y marco conceptual

2.1. Procesamiento documental jurídico

La digitalización de la documentación jurídica permite automatizar numerosas tareas de búsqueda y organización. Sin embargo, existe una diferencia importante entre recuperar información textual y comprender jurídicamente el contenido de un documento.

El presente proyecto se sitúa deliberadamente en el primer ámbito. El sistema identifica patrones previamente definidos y estructura la información resultante. Esta decisión favorece la transparencia: una coincidencia puede rastrearse hasta una regla concreta del programa.

2.2. Expresiones regulares

Las expresiones regulares permiten describir patrones textuales. En el proyecto se emplean para localizar estructuras asociadas a referencias como artículos, reglamentos, directivas y siglas jurídicas.

Su principal ventaja en este contexto es la reproducibilidad. Si un documento contiene el mismo patrón, el sistema aplica la misma regla independientemente de quién ejecute el programa.

Su principal limitación es que la existencia de una coincidencia textual no implica necesariamente que el contexto jurídico sea idéntico.

2.3. Clasificación basada en reglas

La clasificación temática utiliza palabras clave y referencias normativas. El sistema contempla seis grupos:

ID	Tema	Ejemplos de referencias o términos
1	Protección de datos	GDPR, RGPD, Reglamento 2016/679
2	Inteligencia Artificial	AI Act, Reglamento 2024/1689, Artificial Intelligence
3	Competencia	TFEU, TFUE, Reglamento 1/2003, competition
4	Mercados digitales	DMA, DSA, Reglamentos 2022/1925 y 2022/2065
5	Ciberseguridad	NIS2, DORA, Reglamentos 2022/2555 y 2022/2554
6	Resto de temas	Documentos sin coincidencia suficiente con las categorías anteriores

Cuadro 2.1: Categorías temáticas configuradas

2.4. Paralelización

El procesamiento concurrente permite distribuir tareas independientes entre varios usuarios. En este proyecto se utiliza `ThreadPoolExecutor` con un número limitado de usuarios. La estrategia se resume como:

$$W = \text{mín}(4, N)$$

donde N representa el número de documentos disponibles y W el número de usuarios.

La elección de un límite pequeño pretende evitar una proliferación innecesaria de hilos.

Capítulo 3

Metodología

3.1. Diseño experimental

La metodología se organiza en dos grandes fases:

1. extracción de texto;
2. análisis del texto extraído.

Por ello, la separación es fundamental para evitar repetir operaciones de entrada/salida.

3.2. Fase de extracción

Cada PDF se abre mediante PyPDF y se recorren sus páginas. El texto de las páginas se concatena y se normaliza.

El esquema lógico es:

$$PDF_i \rightarrow T_i$$

donde T_i representa el texto extraído del documento i .

3.3. Fase de análisis

Una vez obtenidos los textos, cada documento puede analizarse sin volver a leer el PDF:

$$T_i \rightarrow \{A_i, N_i, S_i, R_i, C_i\}$$

donde:

- A_i : artículos detectados;
- N_i : normativas detectadas;
- S_i : semáforo doctrinal;
- R_i : resumen;
- C_i : clasificación temática.

3.4. Precompilación de expresiones

Las expresiones regulares principales se preparan antes del procesamiento masivo. Esto evita compilar repetidamente los mismos patrones para cada documento. La optimización puede conceptualizarse como:

Compilación única + Reutilización $\rightarrow$ Menor coste repetitivo

3.5. Interfaz gráfica

La interfaz utiliza Tkinter. Las tareas pesadas no se ejecutan directamente dentro del bucle principal de eventos. Los usuarios generan eventos y resultados que posteriormente se procesan en la interfaz.

El patrón empleado puede representarse como:

Worker $\rightarrow$ Queue $\rightarrow$ Tkinter

Esto permite actualizar el progreso sin bloquear la ventana.

3.6. Exportación

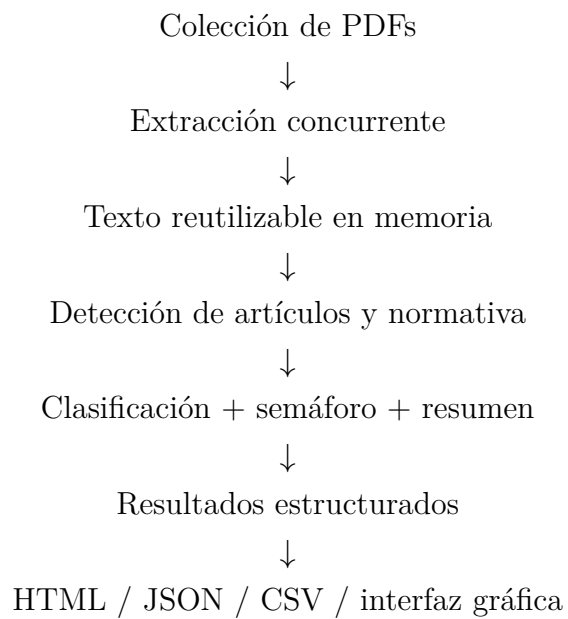
Cada resultado se transforma en estructuras compatibles con tres formatos:

- HTML para consulta visual;
- JSON para intercambio estructurado;
- CSV para análisis tabular.

Diseño e implementación

4.1. Arquitectura

La arquitectura completa puede representarse como:



4.2. Detección de artículos

El sistema busca patrones equivalentes a referencias de artículos. Los resultados se almacenan como una colección estructurada para poder mostrarlos posteriormente en la interfaz.

4.3. Detección normativa

Se emplea un mapa de abreviaturas y denominaciones para reconocer referencias normativas relevantes. Entre ellas aparecen GDPR/RGPD, TFEU/TFUE, DMA, DSA, AI Act, NIS2 y DORA.

4.4. Semáforo doctrinal

La función de semáforo asigna una categoría a partir de la presencia de determinadas referencias.

Estado	Significado en el programa	Ejemplos
Verde	Vigencia absoluta	RGPD, GDPR, AI Act, DSA, DMA, NIS2
Ámbar	Vigencia condicionada	ePrivacy, Directiva 2002/58
Rojo	Referencia superada	Directiva 95/46, Safe Harbour, Privacy Shield, Directiva 2006/24

Cuadro 4.1: Semáforo configurado en el sistema

Debe entenderse estrictamente como una regla programada; no constituye una conclusión jurídica independiente, es decir es necesario la revisión por un profesional del derecho.

4.5. Resumen automático

El sistema intenta identificar cuatro componentes:

1. Partes;
2. Objeto;
3. Doctrina;
4. Fallo.

Cuando no se encuentra un patrón adecuado, el sistema conserva una indicación de que el elemento no ha podido identificarse de forma automática.

4.6. Estructura de datos

La salida de una sentencia sigue una estructura conceptualmente equivalente a:

```
1 {  
2   "documento": "...pdf",  
3   "tema": "...",  
4   "resumen": "...",  
5   "articulos": [...],  
6   "normativas": [...],  
7   "semaforo_color": "VERDE",  
8   "semaforo_emoji": "    ",  
9   "semaforo_etiqueta": "...",  
10  "semaforo_razon": "..."  
11 }
```

Resultados

5.1. Resultados funcionales

La implementación permite completar el flujo previsto: selección de documentos, procesamiento, visualización y exportación.

La interfaz ofrece elementos para seleccionar una carpeta, elegir un tema, añadir PDFs, ejecutar el procesamiento y consultar los resultados. De igual modo, incorpora un contador por temas y botones de acceso a los resultados generados.

5.2. Resultados de rendimiento

El objetivo de la optimización es reducir el coste asociado a repetir la extracción de texto. El diseño rápido puede expresarse como:

$$T_{\text{total}} \approx T_{\text{extracción}} + T_{\text{análisis}} + T_{\text{exportación}}$$

frente a un diseño en el que varias etapas vuelven a abrir cada PDF:

$$T_{\text{repetido}} \approx kT_{\text{extracción}} + T_{\text{análisis}} + T_{\text{exportación}}, \quad k > 1.$$

El valor exacto de la mejora depende del tamaño, estructura y número de PDFs, así como del hardware. Por ello, este documento **no inventa una cifra de aceleración**: para una memoria definitiva debe medirse de forma experimental.

5.3. Resultados de clasificación

La funcionalidad del pipeline es la encargada de auditar y clasificar cuantitativamente el volumen de sentencias procesadas antes o después de su análisis individual.

Dicha aplicación incluye un módulo de auditoría cuantitativa capaz de contabilizar y clasificar las sentencias según su temática jurídica. La salida gráfica resultante permite observar de manera clara e inmediata la distribución del conjunto de documentos entre las seis categorías configuradas:

1. **Protección de datos:** Detección de normativas como el RGPD/GDPR y conceptos clave de privacidad.
2. **Inteligencia Artificial:** Identificación de referencias al *AI Act* y términos afines.
3. **Competencia:** Rastreo de artículos del TFUE (101, 102) y derecho antitrust.
4. **Mercados digitales:** Clasificación basada en la DSA y DMA.
5. **Ciberseguridad:** Identificación de directivas como NIS2 y Reglamento DORA.
6. **Resto de temas:** Categoría residual para sentencias que no cumplen los criterios de las materias anteriores.

El análisis se realiza mediante un motor determinista que escanea el texto extraído de los archivos PDF y calcula la frecuencia de aparición de términos clave predefinidos. La representación gráfica permite al usuario evaluar el sesgo o la concentración temática del *dataset* de forma previa o posterior al procesamiento detallado.

Capítulo 6

Discusión

6.1. Ventajas

Entre las principales ventajas de la solución se encuentran:

- Reproducibilidad;
- Transparencia de las reglas;
- Ausencia de dependencia de un modelo generativo;
- Facilidad de modificación de palabras clave;
- Exportación a formatos estándar;
- Reutilización del texto extraído;
- Interfaz visual para usuarios no técnicos.

6.2. Limitaciones

Las reglas de coincidencia pueden producir falsos positivos o falsos negativos. Por ejemplo, una norma puede aparecer en una sentencia de manera incidental sin que constituya el eje doctrinal del asunto.

Asimismo, un resumen basado en patrones no posee la capacidad de interpretación contextual de un análisis jurídico humano, es por ello que **ha de ser revisado por un profesional**.

La herramienta tampoco incorpora, en el alcance descrito, una validación automática completa de la vigencia de cada norma frente a todas sus modificaciones posteriores.

6.3. Reproducibilidad

La naturaleza determinista del sistema favorece la reproducibilidad. Es decir, en el caso de que se utilicen los mismos documentos, configuración y versión del programa, el procesamiento debería producir resultados equivalentes salvo diferencias derivadas del entorno de ejecución.

Esta propiedad es particularmente útil para construir conjuntos de datos de investigación.

6.4. Utilidad jurídica

La herramienta resulta especialmente adecuada como etapa previa a la lectura detallada. Tras lo anterior, permite pasar de una colección de documentos poco estructurada a un inventario con referencias y categorías.

No obstante, la decisión jurídica final **debe permanecer bajo control humano**.

Capítulo 7

Conclusiones

El proyecto demuestra la viabilidad de construir una herramienta ligera de procesamiento documental jurídico utilizando técnicas deterministas.

La principal aportación técnica de esta versión es la reorganización del pipeline alrededor de una extracción única del texto de cada PDF. La reutilización de esa representación evita operaciones redundantes y permite separar claramente extracción y análisis.

La segunda aportación relevante es el procesamiento concurrente mediante `ThreadPoolExecutor`, combinado con una interfaz Tkinter que recibe eventos mediante una cola. Esta arquitectura permite mantener una interacción razonablemente fluida durante el procesamiento.

Desde la perspectiva jurídica, el sistema ofrece una primera capa de organización documental: artículos, normativa, categorías temáticas, resumen y un semáforo configurable. Su valor reside principalmente en reducir trabajo mecánico y facilitar la revisión posterior.

Como conclusión metodológica, el **sistema debe considerarse una herramienta de apoyo a la investigación jurídica y no un sustituto del razonamiento jurídico.**

Capítulo 8

Trabajo futuro

Por último, se ha de señalar una serie de líneas de desarrollo futuro entre las que se proponen, posteriormente se indican:

1. La Inteligencia Artificial
2. Incorporar OCR para documentos escaneados;
3. Mejorar la detección de estructura de sentencias;
4. Añadir validación frente a fuentes jurídicas oficiales;
5. Medir formalmente precisión, exhaustividad y F1;
6. Comparar distintas estrategias de extracción de PDF;
7. Añadir filtros y búsquedas avanzadas;
8. Incorporar enlaces directos a las sentencias originales;
9. Desarrollar una base de datos persistente;
10. Crear resúmenes estructurados mediante reglas lingüísticas.

Bibliografía

- [1] Python Software Foundation. *Python Documentation*. Documentación oficial del lenguaje Python.
- [2] pypdf contributors. *pypdf Documentation*. Documentación de la biblioteca utilizada para la extracción de texto PDF.
- [3] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. Documentación de la biblioteca gráfica utilizada en el proyecto.
- [4] Python Software Foundation. *re — Regular expression operations*. Documentación oficial del módulo de expresiones regulares.
- [5] Python Software Foundation. *concurrent.futures — Launching parallel tasks*. Documentación oficial de `ThreadPoolExecutor`.
- [6] The pandas development team. *pandas Documentation*. Documentación de la biblioteca utilizada para el tratamiento de datos tabulares.

Apéndice A

Anexo A: Fragmentos principales

A.1. Extracción de texto

```
1 reader = PdfReader(str(pdf_path))
2 partes = []
3
4 for pagina in reader.pages:
5     texto = pagina.extract_text()
6     if texto:
7         partes.append(texto)
8
9 texto_final = " ".join(partes)
```

A.2. Procesamiento concurrente

```
1 with ThreadPoolExecutor(max_workers=workers) as ex:
2     futures = {
3         ex.submit(extraer_texto, pdf): pdf
4         for pdf in pdfs
5     }
```

A.3. Procesamiento desde texto reutilizado

```
1 def analizar(item):
2     pdf, texto = item
3     return pdf, procesar_documento_desde_texto(
4         pdf, texto, tema_id
5     )
```